

Spotify Import Failure Diagnostic & Repair Plan

Spotify Import Failure Diagnostic & Repair Plan

This document provides a complete, structured plan for diagnosing and repairing the Spotify import failure in the Audio Scrobbler App. It is based on observed symptoms: missing backend routes, inability to log into Spotify, missing ingestion worker container, and Podman-related networking issues. The goal is to restore full Spotify OAuth functionality and enable ingestion of listening history.

1. Verify Backend Routes Exist

The backend must expose the following routes:

- `/auth/me`
- `/spotify/login`
- `/spotify/callback`
- `/ingestion/trigger`

Required Actions

- Search the backend codebase for these route definitions.
- Confirm that each route is included in the main FastAPI application via `app.include_router(...)`.

Why This Matters

If these routes are missing or not mounted, the PWA cannot authenticate with Spotify, cannot receive the OAuth callback, and cannot trigger ingestion.

2. Start Backend Using Compose

The backend must be started using the compose stack to ensure all dependent services are created correctly.

Required Actions

- Start the stack using Docker Compose or Podman Compose.
- Confirm that backend, postgres, ingestion-worker, and secrets are all running.

Why This Matters

Running the backend manually bypasses router mounting, environment variables, secrets, and worker startup. This leads to missing endpoints and failed ingestion.

3. Verify Ingestion Worker Container Exists

The ingestion worker is responsible for fetching Spotify listening history.

Required Actions

- Inspect the compose file for an ingestion-worker service.
- Ensure Podman has started the worker.

Why This Matters

Without the ingestion worker, Spotify ingestion cannot run even if OAuth succeeds.

4. Validate Spotify Callback Endpoint

The Spotify OAuth callback must be reachable.

Required Actions

- Test the callback endpoint using `curl -I http://localhost:8000/spotify/callback`.
- If the endpoint returns 404, fix router mounting.

Why This Matters

If the callback endpoint does not exist, Spotify cannot return the authorization code. This prevents login and blocks ingestion entirely.

5. Confirm PWA Backend URL

The PWA must point to the correct backend URL.

Required Actions

- Check the PWA environment configuration.
- Ensure `VITE_API_URL` matches the backend host and port.

Why This Matters

If the PWA calls the wrong host or port, it will show "Failed to fetch" even if the backend is working.

6. Re-run Full Spotify OAuth Flow

Once backend and callback are correct, the OAuth flow must be completed.

Required Actions

1. Click **Login with Spotify** in the PWA.
2. Complete Spotify login.
3. Backend stores encrypted refresh token.
4. Ingestion worker begins fetching listening history.

Why This Matters

You have never logged into Spotify on the localhost app. Without a refresh token, ingestion cannot start.

7. Trigger Ingestion Manually

Manual ingestion verifies the pipeline end-to-end.

Required Actions

- Send a POST request to `/ingestion/trigger`.
- Inspect ingestion worker logs.

Why This Matters

This confirms that ingestion is functioning once OAuth is fixed.

Summary of Root Causes

- Spotify OAuth has never completed.
- Backend routes for authentication and ingestion are missing or not mounted.
- Ingestion worker container is not running.
- Podman networking and secrets may be misconfigured.
- PWA is unable to reach backend routes.

These issues collectively prevent Spotify login, token storage, and ingestion.

Next Steps

Follow the plan step-by-step to restore full Spotify import functionality. Once backend routes, worker container, and callback endpoint are corrected, the OAuth flow will succeed and ingestion will operate normally.